

Overview
Lecture notes
Assignmen
Submission server
Forum

Garbage collection

Introduction

Your goal in this final assignment is to implement a mark-and-sweep garbage collector for the L_3 virtual machine, L_3 VM.

Initial setup

To start working on the assignment, you should once again download [the Zip archive containing the skeleton](#), and unzip it in your project.

Overview

After extracting the contents of the archive, you will have access to the following new code:

1. the phases required to generate assembly code for L_3 VM, and
2. a partial version of the VM, to be completed for this assignment.

There are two versions of the VM: one written in C, the other in Rust. You must choose one of the two VMs to complete, and ignore the other.

C VM

The C VM is in the `vm/c` subdirectory. A `Makefile` is given to you, which you can use to build the VM.

The `Makefile` assumes that you will compile your code with the `clang` compiler, which we strongly recommend you do, as it provides two “sanitizers” that help tremendously during debugging:

- the [address sanitizer](#), which detects a lot of common memory-related errors,
- the [undefined behavior sanitizer](#), which detects a lot of errors related to C’s (unfortunately many) undefined behaviors.

By default, both sanitizers are enabled, and we encourage you to keep them during development, by setting the `CFLAGS` variable of the `Makefile` to the value of `CFLAGS_DEBUG`. Once you’re done

debugging, you can use the value of `CFLAGS_RELEASE` instead, to see how fast the test programs run.

Rust VM

The Rust VM is in the `vm/rust` subdirectory. It can be built as usual with the `cargo` command. To build the debug version, simply type `cargo build`, and to build the faster release version, type `cargo build --release`.

To run one of these versions, and to build it before if needed, type `cargo run` with or without the `--release` flag depending on the version you want to run.

L₃VM

(Note: only the C version of the VM is described below, but the Rust one is very similar.)

The L₃VM contains two major components:

1. the (threaded) interpreter, in `engine.c`,
2. the memory manager, in `memory.c`.

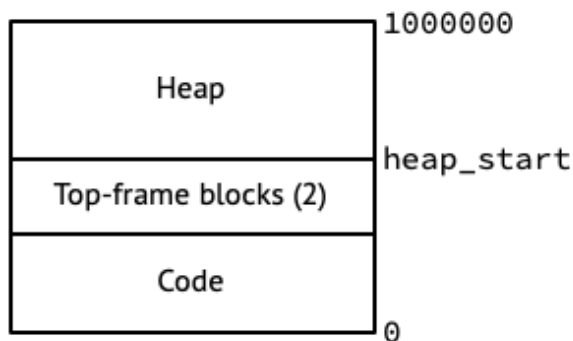
The current version of the memory manager allocates but never frees memory. It can therefore run programs with small inputs correctly, but fails on bigger inputs. For example, it can compute the factorial of 40 using the `bignums` test:

```
$ cd vm/c
$ make
(output elided)
$ ./bin/vm ../test/bignums.l3a
Factorial of? 40
40! = 8159152832478977343456112695961158942720000000000
```

As described in the lecture, the VM uses a (virtual) 32-bit address space. Its total size, in bytes, is specified on startup using the `-m` option. By default, it is 1 000 000, i.e. one megabyte.

When the VM starts, it first loads the code at the beginning of the address space, starting at 0, then adds the two top-frame blocks after it, and finally uses the rest of the memory for the heap.

Assuming the default memory size, the address space therefore looks as follows:

Figure 1: L₃VM address space

Internally, the VM manipulates two kinds of addresses:

- *virtual addresses*, which have type `value` and are therefore 32 bits, refer to the address space shown above, e.g. address 0 is that of the first instruction,
- *physical addresses*, which have type `value*` and are therefore 32 or 64 bits pointers (depending on the host machine), refer to the address space of the host machine on which the VM is running.

Don't be misled by the "physical" qualifier of the second kind of addresses: they are physical only from the point of view of the VM. From the point of view of the host operating system, they are virtual addresses, but this does not concern us here.

As you will see by looking at the code, the VM uses a mixture of physical and virtual addresses, for performance reasons. For example, the interpreter maintains a physical pointer to the beginning of the registers of the current function, but stores virtual pointers in the heap, of course.

The garbage collector

The goal of this assignment is to write a version of the memory manager that collects unreachable blocks using a mark-and-sweep algorithm.

We suggest you use the current version of the memory manager as your starting point, as it already handles some aspects correctly, e.g. the packing and unpacking of a block's size and tag in its header.

Notice that you'll often have to translate between physical and virtual addresses, and for that you can use the `addr_v_to_p` and `addr_p_to_v` functions from `address.h`.

Bitmap

Since untagged integers can be present in activation frames, and since those frames can be evicted to the heap, the GC can encounter untagged integers during its marking phase. These untagged integers can look like valid heap pointers, and blindly following and interpreting the memory they “point” to as if it was a valid object leads to problems.

To avoid this, the garbage collector must be able to know, given a heap address, whether an allocated block starts at that address or not. To that end, it maintains a bitmap with one bit per word of the heap. When a block is allocated, the bit corresponding to its first word is set in the bitmap.

During the marking phase, the GC can check the bitmap to make sure that a value that looks like a pointer really refers to the beginning of an allocated block, and only follow it in that case. Notice that this doesn't mean that the value is actually a pointer, it could very well be an untagged integer, but at least the GC never treats some random part of the heap like an allocated block, which is what leads to serious problems.

Surprisingly, the bitmap can also be used to store the mark bits during the marking phase! To understand how, notice that at the beginning of the marking phase, all allocated blocks (and only them) have their bit set in the bitmap. Therefore, during the marking phase, the GC can *clear* the bit corresponding to a block to mark it. That way, at the end of the marking phase, the bits that are 0 correspond to either:

1. allocated and reachable blocks, or
2. unallocated blocks, i.e. those that were already in the free list when GC started.

Provided that unallocated blocks can be identified, for example with a special tag reserved for them (`tag_FreeBlock` in `memory.h`, which is 0), it is then easy for the sweep phase to add them to the free list, along with unreachable blocks.

Notice that the bitmap must be stored in the same memory as the one used for the code and the heap! Therefore the address space of the VM looks as follows if one includes the bitmap (the position of the heap and bitmap could also be swapped):

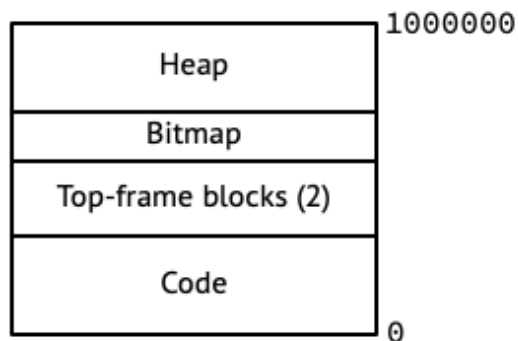


Figure 2: L3VM address space (with bitmap)

The size of the bitmap must be 1/32th of the size of the heap, rounded up to the next integer. Beware that correctly computing that size is not completely trivial!

Free list

The blocks of memory that are currently free must be stored in some data structure, known as the *free list*, in which the allocator looks for blocks to satisfy allocation requests.

The simplest data structure one can use for the free list is a unique, singly-linked list containing all the free blocks. Notice that the link from a free block to the next can be stored in the block itself, since it is not used by the program.

Beware however that blocks of size 0 can be problematic, as they don't have enough space to store the link to the next block! This can be solved either by making sure that even these blocks have space for the link, or by never storing blocks of size 0 in the free list, waiting instead for them to be coalesced with adjacent free blocks.

While a unique free list works, it is not very efficient as the allocator has to iterate over its elements on each allocation request, in order to find a free block of an appropriate size.

A better solution is to have several *segregated free lists*, e.g. 32 of them. The list at index i contains only blocks of size i , except for the last one, which contains all blocks that do not fit elsewhere. This allows allocation to be very fast in the case where the program requests a block of a size that has a dedicated free list, and that free list is not empty.

Roots

At first glance, it might seem that the roots of the reachability graph consist only of the frame pointer, which points to one of the two top-frame blocks. And this is *almost* correct, but there is one small snag: if the frame of the caller of the currently-executing function resides in the other top-frame block, then by default the garbage collector will ignore it, since the pointer to it — located in the second slot of the top-frame block referenced by the frame pointer — is pointing outside of the heap.

For that reason, it is necessary to handle this as a special case and explicitly mark the second top-frame block — the one that is not referenced by the frame pointer — if it happens to contain the frame of the caller of the currently-executing function.

Marking

The marking phase can be written either recursively, or iteratively, by maintaining an explicit stack of pointers to follow.

The iterative solution is more robust as it consumes less memory and does not risk overflowing the stack. However, it requires (re)allocating memory dynamically, which is generally disallowed (see [Dynamic memory allocation](#) below), but exceptionally allowed in this case.

Splitting and coalescing

During allocation, blocks that are too big compared to the requested size must be split. Beware however of not creating blocks that are too small to be put back in the free list while splitting!

During the sweeping phase, adjacent free blocks must be coalesced into a single, bigger block.

Dynamic memory allocation

Your garbage collector is only allowed to dynamically allocate memory in two cases:

1. at initialization, when allocating the big block that will contain the code, the top-frame blocks, the heap, etc.
2. during the marking phase, if it is done iteratively, to store the marking stack.

Beware that this constraint means that you **cannot**, in general, use data-structures from libraries to implement your garbage collector, as they all allocate memory dynamically.

Tips and tricks

Debugging garbage collectors is tricky, as you're about to find out. Therefore, make sure that you think — a lot, and clearly — before writing code!

Also, sprinkle your code with assertions, using the `assert` macro, to check all the invariants that you can think of. Don't hesitate to write specific functions that check the most important ones, e.g. that the bitmap is consistent with the contents of the heap.

Finally, be careful not to write code that depends on specific characteristics of the platform on which you test it!

Testing and submission

To test your garbage collector, you can use the compiled programs given in the `vm/tests` subdirectory, or compile and run your own tests.

To run the tests, you should install [shelltestrunner](#) on your computer. Once this is done, place yourself in the directory containing the tests and launch the following command:

```
$ shelltest -D VM=../c/bin/vm *.test
```

to test the C VM, or:

```
$ shelltest -D VM=../rust/target/release/l3vm *.test
```

to test the (release version of the) Rust VM.

Notice that all of the tests but 5 are so small that they pass even without a GC! What's hard is making those 5 “big” tests work.

For 4 of the 5 “big” tests that are given, the table below shows the input values that should work correctly both without a GC — using the original `memory.c` module — and with a mark-and-sweep GC, always using the default memory size of 1 Mb. The time taken by our reference C and Rust implementations, which use segregated free lists, to execute the benchmarks on a laptop from 2020 is also shown and should give you an idea of what to aim for.

Test	no GC	M&S GC	C time (s)	Rust time (s)
bignums	296	8000	1.5	2.6

Test	no GC	M&S GC	C time (s)	Rust time (s)
maze	10; 1	55; 1	1.7	3.0
queens	11	18	1.6	3.0
unimaze	75; 15; 1	80; 80; 1	0.04	0.07

Submission is done as usual through [the submission server](#). Notice that this time you have two tokens at your disposal, and you have to use the right one depending on the version of the VM you want to submit. The command to use to create the archive also differs.

If you worked on the C VM, then create the archive as follows:

```
$ cd vm/c
$ zip -r l3vm-c.zip src
```

and submit the Zip archive created by the second command (`l3vm-c.zip`).

If you worked on the Rust VM, then create the archive as follows:

```
$ cd vm/rust
$ zip -r l3vm-rust.zip Cargo.toml src
```

and submit the Zip archive created by the second command (`l3vm-rust.zip`).

Grading

A total of 100 points are assigned to this project part, out of a total of 500 for the whole semester.

You can choose to use either a single free list, or segregated free lists in your garbage collector. Notice however that only doing the former will not allow you to get all the points, as explained below.

Keep in mind that a working GC with a single free list, which is easier to write, is better than a crashing GC with segregated free lists. Therefore, we suggest you start implementing a single free list, submit that version when it works, and only then start working on segregated free lists.

Your submission will be graded based on:

1. its correctness, i.e. whether it passes all the functional tests that were made available to you, and additional ones that we

might have,

2. its completeness: using a single free list instead of segregated lists will cost you 15 points, and using the default, empty `memory_free_block` that does nothing instead of putting the block back to the free list will cost you 5 points more,
3. its performance, which should be within an order of magnitude of our reference implementation,
4. its style, so please pay attention to the feedback you received for the previous assignments.